Introduction to Robotics: ROS 2 Lab. 3

ROS 2 Communication: Topics and Services

Professor: Pietro Falco
Tutor: Alessandro Adami, email: adamialess@dei.unipd.it

Department of Information Engineering (DEI)
Università degli Studi di Padova

March 9, 2026

1. The ROS 2 API: Interface Definitions
2. ROS 2 Communication
3. ROS 2 Topics
4. ROS 2 Publisher
5. Robotic Concepts: The Pose
6. ROS 2 Subscriber
7. ROS 2 Publisher-Subscriber closed loop
8. Verification and Debugging
9. ROS 2 Services - Introduction
10. ROS 2 Services
11. Hands-on

The ROS 2 API: Interface Definitions

Before nodes can communicate, they must agree on a **Contract**. In ROS 2, this contract is called an **Interface**.

- **Strict Typing**: Unlike standard Python, ROS 2 communication is strictly typed. You cannot send a "String" to a topic expecting an "Integer."
- **Language Neutral**: The API is defined in simple text files (`.msg`, `.srv`), which the ROS 2 build system translates into Python classes automatically.
- **Standardization**: Official APIs (e.g., docs.ros2.org) specify the **Data Type** and **Units** (always S.I. units: meters, radians, seconds).

API Components

Interface Type	File Extension	Communication Pattern
Message	`.msg`	Topics (Continuous Data Streams)
Service	`.srv`	Services (Request/Response Handshake)
Action	`.action`	Actions (Goal-Oriented with Feedback)

Why start here?

If you don't follow the API exactly, your nodes will refuse to talk. **The API is the "Dictionary" of your robotic system.**

You don't need a web browser to check an API. You can "interrogate" any message type directly from your terminal inside Docker.

Using the CLI to see the Contract

```
# Syntax: ros2 interface show <package_name>/msg/<MessageName>

# 1. Check a simple integer message
ros2 interface show std_msgs/msg/Int32
# Result: int32 data

# 2. Check a complex movement message
ros2 interface show geometry_msgs/msg/Twist
# Result: Vector3 linear, Vector3 angular...
```

Key Concept

When you see `float64 x` in the API, it tells you exactly how to access it in Python: `msg.x`. If you see `Vector3 linear`, it's a nested object: `msg.linear.x`.

If your code crashes with a `TypeError` or `AttributeError`, follow these steps:

1. **Check the Interface**: Run `ros2 interface show <type>`. Are you using `linear.x` or just `x`?
2. **The Nesting Rule**: In the CLI, if a field is indented under another (like `x` under `Vector3 linear`), you **must** use dot notation: `msg.linear.x`.
3. **The Array Rule**: If the API shows `float32[]`, the variable is a List. Access it with `msg.ranges[0]`, not `msg.ranges`.

Common Pitfall

Python is flexible, but ROS 2 messages are not. If the API says `float64`, do not try to send a `string` "2.0". Use the number `2.0`.

ROS 2 Communication

In ROS2, we don't build a single, massive program. Instead, we build a Computational Graph.

- **Decentralized Architecture:** Each task (sensing, planning, acting) is handled by a separate process called a **Node**.
- **Fault Tolerance:** If the camera node crashes, the motor control node can still trigger an emergency stop.
- **The Middleware (DDS):** Data flows between nodes through an underlying software layer that handles the network "plumbing."

Key Philosophy

Small, modular nodes are easier to debug, reuse, and share with the robotics community than one giant script.

ROS2 provides three distinct ways for nodes to interact. Choosing the right one depends on the nature of the data.

- **Topics (Continuous Flow):** Used for data streams. High-frequency, one-way communication.
- **Services (Request/Response):** Used for quick, discrete commands. The client waits for the server.
- **Actions (Goal-Oriented):** Used for long tasks (e.g., "Drive to the kitchen"). Includes feedback on progress and the ability to cancel.

Which one to use?

Scenario	Paradigm	Example
Reading Laser Data	**Topic**	`/scan`
Resetting an Encoder	**Service**	`/reset_odometry`
Autonomous Navigation	**Action**	`/navigate_to_pose`

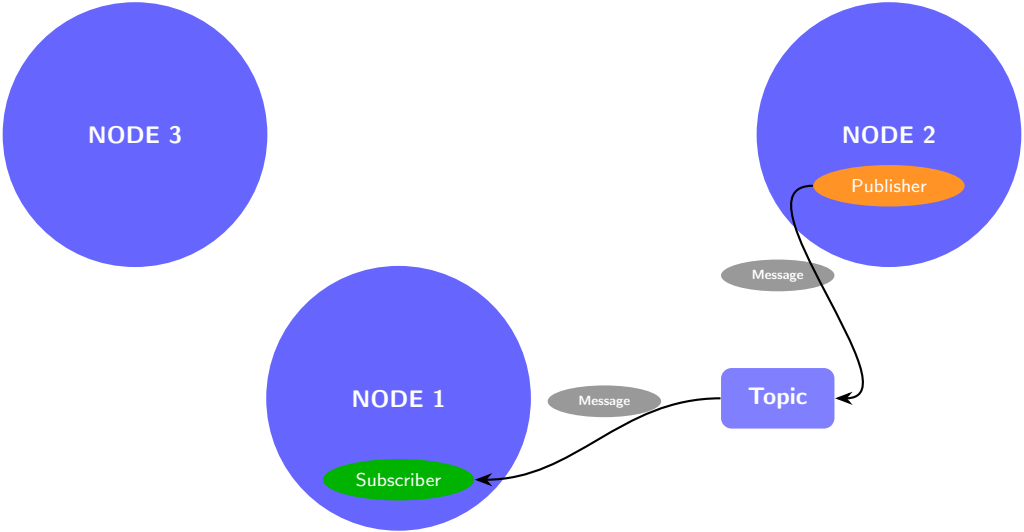
ROS2 has no "Master." Nodes find each other automatically using DDS (Data Distribution Service).

1. **Broadcast:** When a node starts, it announces its presence on the network.
2. **Matching:** If Node A publishes a topic called `/cmd_vel` and Node B wants to subscribe to `/cmd_vel`, the middleware links them.
3. **Interoperability:** This works even if one node is written in Python and the other in C++.

Network Isolation

You can isolate groups of nodes by setting a `ROS_DOMAIN_ID`. Nodes with different IDs will not "see" or talk to each other.

ROS 2 Topics


NODE 3
NODE 2
Publisher
Message
NODE 1
Message
Topic
Subscriber

Topics are a simple way for nodes to communicate, since they don't "talk" directly. Topics follow a **Publish/Subscribe** model, and they interact through a chatter.

- **Publisher**: Publish a message to the chatter
- **Subscriber**: Subscribe and read to the chatter

It is **anonymous**: the Publisher doesn't know who is listening, and the Subscriber doesn't know who is sending (the user can).

- **Bus Analogy**: It's like a specific radio frequency. Anyone can tune in, and anyone with a transmitter can broadcast.
- **Message Types (.msg)**: Every topic has a strict data structure (e.g., `sensor_msgs/msg/LaserScan`).
- **Frequency**: Topics usually run at a fixed rate (e.g., 10Hz, 50Hz).

Terminal Debugging

```
# 1. See what topics are active
ros2 topic list
# 2. See the data in real-time
ros2 topic echo /cmd_vel
# 3. Check the frequency of the data
ros2 topic hz /odom
# 4. Check info about the topic (type, publishers, subscribers...)
ros2 topic info /cmd_vel
```

To communicate, nodes must agree on the **Interface**. You cannot send a Python List directly; you must wrap it in a ROS Message.

- **Standard Messages**: `std_msgs` (Int32, String, Float64).
- **Sensor Messages**: `sensor_msgs` (Image, LaserScan, Imu).
- **Geometry Messages**: `geometry_msgs` (Twist, Pose, Transform).

Inspecting a Message in the Terminal

```
# Syntax: ros2 interface show <message_type>
ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into linear and angular parts.
Vector3  linear
        float64 x
        float64 y
        float64 z
Vector3  angular
        float64 x
        float64 y
        float64 z
```

Common Mistake

If you try to publish a `float` to a topic expecting a `std_msgs/msg/Float64`, your node will crash. You must import the type: `from std_msgs.msg import Float64`.

A `geometry_msgs/msg/Twist` handles 6 degrees of freedom, but for a 2D robot (like the Turtle), we only use two:

- **linear.x**: Forward/Backward velocity (in m/s).
- **angular.z**: Rotation speed (in rad/s). Positive is Counter-Clockwise.

Handy Conversion

Remember: 2π radians $= 360^\circ$. If you set `angular.z = 1.57`, the turtle turns 90° per second.

ROS2 uses a **Discovery Protocol** (via DDS). When a node starts, it "shouts" on the network: *"I am Node A, and I publish Topic X!"*

1. **The Participant**: Every process in ROS2 is a DDS Participant.
2. **The Match**: If Node B says *"I want to subscribe to Topic X"*, the middleware connects them automatically.
3. **Domain ID**: By default, ROS2 nodes only see others on the same `ROS_DOMAIN_ID` (default is 0).

Lab Connectivity advanced tips (we will not use this)

Inside Docker, if you run two containers on the same virtual network, they will "discover" each other automatically. This allows you to run the simulator in one container and your controller in another!

In a real robotic "brain," a single node often needs to listen to multiple sensors and publish various commands simultaneously.

- **Data Fusion:** A node can subscribe to both `/scan` (Lidar) and `/odom` (Odometry) to navigate safely.
- **One-to-Many:** One topic (e.g., `/clock`) can be consumed by every node in the system.
- **Many-to-One:** Multiple nodes can publish to `/logs`, but be careful with actuator topics like `/cmd_vel`!

Design Tip: Modularity

Nodes are decoupled. You can add a new Subscriber (e.g., a data logger) to an existing Topic without stopping or modifying the Publisher.

A **LiDAR** (Light Detection and Ranging) is a "Time-of-Flight" sensor (range-finder sensor) that measures distance by hitting a target with a laser pulse and measuring the time it takes for the reflection to return.

- **Scanning**: 2D Lidars (common in mobile robots) spin a single laser beam 360° to create a 2D "slice" of the environment.
- **Data Type**: In ROS 2, this is represented by the `sensor_msgs/msg/LaserScan` message.

The LaserScan Structure

```
# Use: ros2 interface show sensor_msgs/msg/LaserScan
float32 angle_min          # Start angle of the scan [rad]
float32 angle_increment    # Angular distance between measurements
float32[] ranges           # The actual distance data (A Python List!)
```

Placeholder: Lidar Visualization

In RViz (the ROS 2 Visualizer), a Lidar scan looks like a collection of dots forming the "walls" of the room.

Robotics involves hundreds of message types. Instead of memorizing them, you must learn to read the **API Documentation**.

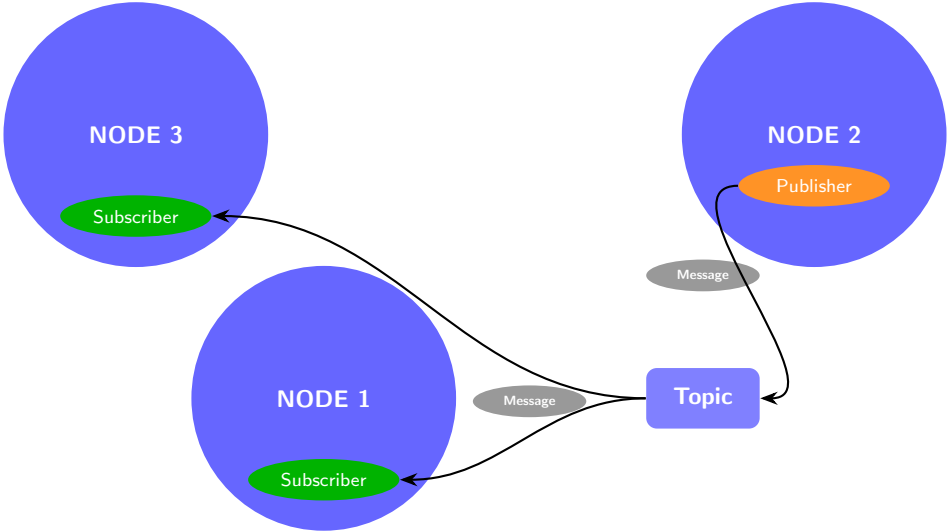
- **Online Reference**: Official docs (like **docs.ros2.org/latest/api/sensor_msgs/msg/LaserScan**) provide the definitive structure.
- **Field Definitions**: The API tells you the **Unit** (e.g., meters, radians) and the **Type** (e.g., `float32[]`).

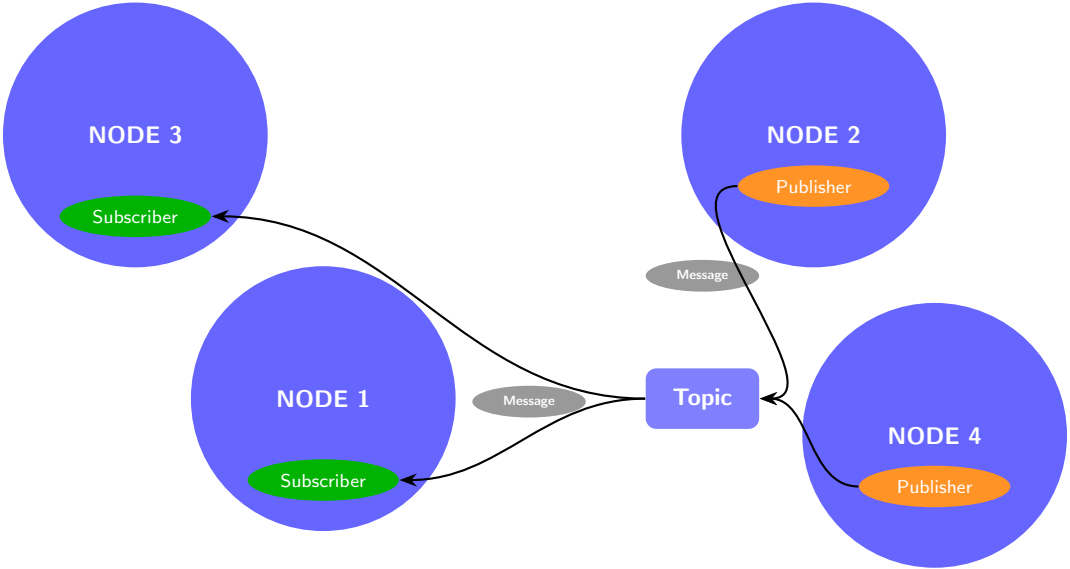
Anatomy of a LaserScan API

Field	Type	Meaning
`header`	`Header`	Timestamp and Frame ID (Where is the sensor?).
`angle_min`	`float32`	The start angle of the scan [rad].
`ranges`	`float32[]`	**Array** of distances from the sensor [m].
`intensities`	`float32[]`	Strength of the return (useful for detecting materials).

Placeholder: API Screenshot

If you see `[]`, it means the variable is a **List**. In Python, you can use `len(msg.ranges)` to see how many "laser beams" your sensor has.


NODE 3
Subscriber
NODE 2
Publisher
Message
NODE 1
Message
Topic
Subscriber

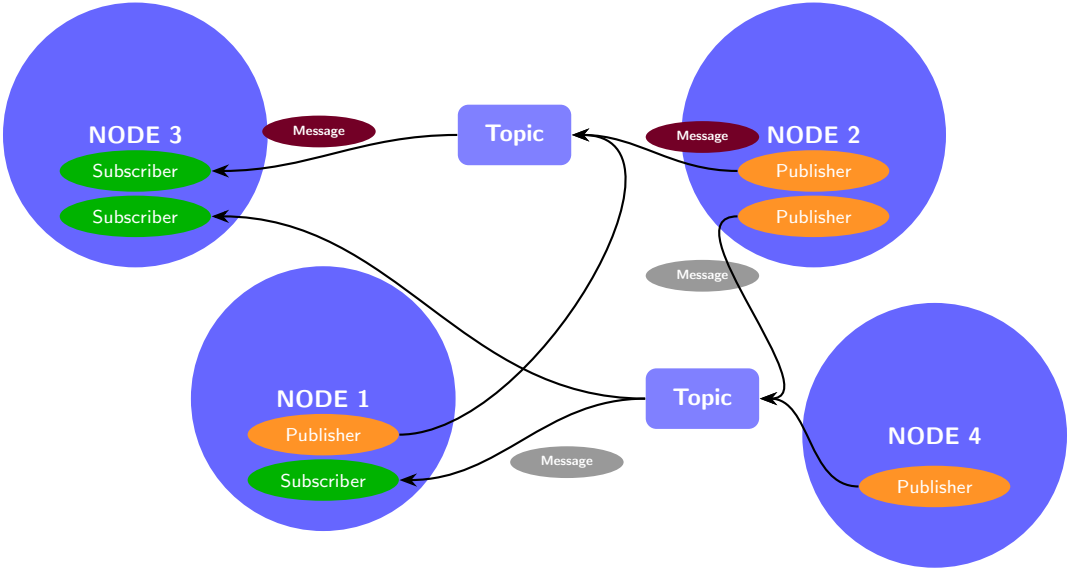

NODE 3
Subscriber
NODE 2
Publisher
Message
NODE 1
Subscriber
Message
Topic
NODE 4
Publisher

When your system grows to include multiple connections, use these tools to verify the "plumbing" of your robot.

- **Trace Connections:** See which nodes are talking on a specific topic. `ros2 topic info /cmd_vel --verbose`
- **Node Audit:** List every Publisher and Subscriber hosted by a single node. `ros2 node info /multi_io_node`
- **Visual Graph:** Generate a live map of all active connections. `ros2 run rqt_graph rqt_graph`

The "Shadow" Publisher

If two nodes publish to `/cmd_vel` at once, the robot will "stutter" as it tries to follow two different paths. Always check `topic info` to ensure only one controller is active!


NODE 3
Subscriber
Subscriber
Message
Topic
Message
NODE 2
Publisher
Publisher
Message
NODE 1
Publisher
Subscriber
Topic
Message
NODE 4
Publisher

To use a message in Python, you must understand its underlying structure. A `Twist` message is actually composed of two `Vector3` messages.

Message Definition (.msg):

geometry_msgs/msg/Twist

```
Vector3 linear
Vector3 angular
Inside Vector3.msg:
float64 x
float64 y
float64 z
```

Accessing in Python:

- `msg.linear.x`
- `msg.linear.y`
- `msg.linear.z`
- `msg.angular.x`
- `msg.angular.y`
- `msg.angular.z`

Key Concept: Types

Notice that the primitive types are `float64`. In Python, these map directly to standard `float`, but in C++, they would be `double`.

- **Exploration Tool:** You can inspect any message on your system using the command line:

```
ros2 interface show geometry_msgs/msg/Twist
```

ROS 2 Publisher

As we did in the previous laboratory, create a new script `touch draw_circle.py` and make it executable in your package.

A Minimal Publisher Node Template

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
class DrawCircleNode(Node):

    def __init__(self):
        super().__init__("draw_circle")
        self.cmd_vel_pub_=self.create_publisher(Twist,"/turtle1/cmd_vel",10)
        self.timer_=self.create_timer(0.5, self.send_velocity_command)
        self.get_logger().info("Draw circle node has been started")

    def send_velocity_command(self):
        msg = Twist()
        msg.linear.x = 2.0
        msg.angular.z = 1.0
        self.cmd_vel_pub_.publish(msg)

def main(args=None):
    ...
```

This time, we import a specific type of message

```
from geometry_msgs.msg import Twist
```

Now we create the publisher, the core of this example node. We specify the type of the message, the exact name, and the size of the message. With the timer, then we send data.

```
        self.cmd_vel_pub_=self.create_publisher(Twist,"/turtle1/cmd_vel",10)
        self.timer_=self.create_timer(0.5, self.send_velocity_command)
```

turtle1

Turtlesim is a great tool, but it's "picky." If you name your topic /cmd_vel but Turtlesim is listening on /turtle1/cmd_vel, nothing happens.

Creating the Publisher

```
self.cmd_vel_pub_ = self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
```

- **Type**: `Twist` defines the data structure.
- **Topic**: `"/turtle1/cmd_vel"` is the specific "bus" name.
- **History Depth (10)**: This is the Queue Size. If the subscriber is slow, ROS 2 buffers up to 10 messages. When the 11th message arrives, the oldest is dropped. This ensures the robot always receives the most "fresh" data.

Timing the Execution

```
self.timer_ = self.create_timer(0.5, self.send_velocity_command)
```

- This tells the node to trigger the callback every **0.5 seconds** (2 Hz).
- It is the "heartbeat" of your node, ensuring commands are sent continuously without manual loops.

Logging Information

```
self.get_logger().info("Message sent!")
```

- Instead of using `print()`, we use **Loggers**. They categorize messages (INFO, WARN, ERROR) and allow you to view them in the terminal or save them to a file for later debugging.

Inside the Callback: Creating and Sending Data

- **Object Instantiation**: `msg = Twist()` creates a local instance of the message. By default, all numerical fields (x,y,z for both linear and angular) are initialized to **0.0**.
- **Field Access**: We use "dot notation" to navigate the message hierarchy. Since a `Twist` contains two `Vector3` objects, we access them via `.linear` and `.angular`.

```python
def send_velocity_command(self):
    msg = Twist() # 1. Initialize empty message
    msg.linear.x = 2.0 # 2. Set forward velocity (m/s)
    msg.angular.z = 1.0 # 3. Set rotation velocity (rad/s)
    self.cmd_vel_pub_.publish(msg) # 4. The "Hand-off"
```

The `.publish(msg)` Function

- **Serialization**: This function converts your Python object into a raw stream of bytes that can be sent over a network.
- **DDS Hand-off**: It hands the data to the underlying **DDS middleware**, which ensures the message reaches all compatible subscribers currently active in the "Graph."

Default Values

You only need to assign values to the fields you want to change. Everything else (like `msg.linear.y`) remains zero, which is a default safe for robot movement.

As we imported the Twist message from the ROS 2 `geometry_msgs` library, we may add dependencies in our package. In `package.xml` add

```
<depend>geometry_msgs</depend>
```

setup.py

Install the node in console scripts as we did before in `setup.py`. Remember to run `colcon build --symlink-install` every time `setup.py` is updated.

Figure: Graph with publisher. If the node stops publishing, the turtle stops.

Robotic Concepts: The Pose

In robotics, knowing only the "position" of a robot is usually not enough. We need to know its **Pose**.

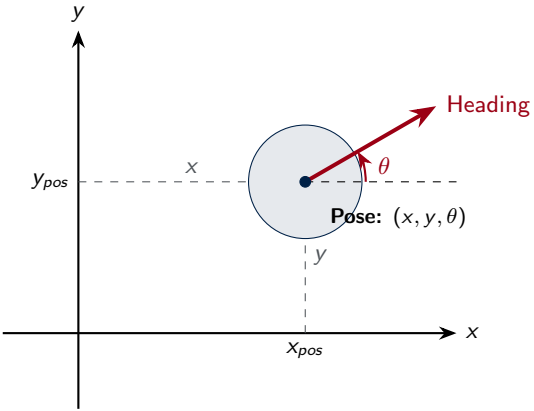
- **Position**: The coordinates (x, y, z) in a reference frame (the "Where").
- **Orientation**: The direction the robot is facing (the "Heading").

The 2D Case (Turtlesim) Since the turtle moves on a flat screen, its pose is simplified:

- **Position**: x and y coordinates.
- **Orientation (θ)**: The angle in radians relative to the x-axis.

Pose vs. Position

If a robot is at $(0, 0)$, it could be facing North, South, East, or West. To plan a path, we must know the **full Pose** $[x, y, \theta]^T$.



Coordinate Frames

In ROS 2, the origin $(0, 0)$ is usually the bottom-left of the window. The angle $\theta = 0$ points along the positive x-axis (to the right).

ROS 2 Subscriber

As we did in the previous laboratory, create a new script `touch pose_subscriber.py` and make it executable in your package.

A Minimal Publisher Node Template

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from turtlesim.msg import Pose

class PoseSubscriberNode(Node):

    def __init__(self):
        super().__init__("pose_subscriber")
        self.pose_subscriber_=self.create_subscription(Pose,"/turtle1/pose",
self.pose_callback, 10)

    def pose_callback(self, msg: Pose):
        self.get_logger().info("(" + str(msg.x) +", " + str(msg.y) +
")")

def main(args=None):
    . . .
```

This time, we import a specific type of message from the `turtlesim` package

```
from turtlesim.msg import Pose
```

> **`setup.py` and `package.xml`**
>
> Remember to update `setup.py` and `package.xml`

The Subscription: Listening to the Network We specify the type of the message, the exact name, and the size of the message buffer

```
self.pose_subscriber_ = self.create_subscription(Pose,
"/turtle1/pose", self.pose_callback, 10)
```

- **No Timer?**: Unlike the Publisher, we don't decide when to run. We are **reactive**. The node sleeps until a message arrives on the topic.
- **Event-Driven**: The arrival of data is the "event" that wakes up the node.
- **Callback (`self.pose_callback`)**: This is the function name we "register" with ROS 2. We are telling the middleware: *"Whenever you see a Pose message, automatically run this function for me."*

The Callback Function

```
def pose_callback(self, msg: Pose):
    self.get_logger().info(f"Position: ({msg.x:.2f}, {msg.y:.2f})")
```

- **The `msg` Argument**: ROS 2 automatically passes the received data into this variable. You don't call this function yourself; the `rclpy.spin()` executor does it for you.
- **Data Access**: Since we imported `Pose`, we can access fields like `msg.x` and `msg.theta` directly.

Why no Timer?

If you used a timer to read data, you might check too fast (reading the same data twice) or too slow (missing messages). By using a Subscriber Callback, you process every single piece of data when it arrives.

As we imported the a new message from the `turtlesim.msg` library, we may add dependencies in our package. In `package.xml` add

```
<depend>turtlesim</depend>
```

setup.py

Install the node in console scripts as we did before in `setup.py`. Remember to run `colcon build --symlink-install` every time `setup.py` is updated.

Figure: Graph with both publisher and subscriber. If the node stops publishing, the turtle stops, but the subscriber still reads the other message.

ROS 2 Publisher-Subscriber closed loop

Up until now, the subscriber and the publisher were two separate nodes, but a node can contain both. For example, to command the turtle we need pose and send control velocities and, an unique node can do this.

A Minimal Publisher-Subscriber Node Template

```python
class TurtleControlNode(Node):

    def __init__(self):
        super().__init__("turtle_controller")
        self.pose_subscriber_=self.create_subscription(Pose,"/turtle1/pose", self.pose_callback, 10)
        self.cmd_vel_publisher_=self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
        self.get_logger().info("Turtle controller has been started.")


    def pose_callback(self, pose: Pose):
        cmd = Twist()
        if pose.x > 9.0 or pose.x < 2.0 or pose.y > 9.0 or pose.y < 2.0:
            cmd.linear.x = 1.0
            cmd.angular.z = 0.9
        else:
            cmd.linear.x = 5.0
            cmd.angular.z = 0.0
        self.cmd_vel_publisher_.publish(cmd)
```

In our previous nodes, we used a **Timer** to "clock" our code. In this closed-loop controller, the **Data** itself becomes the trigger.

- **The Subscriber is the Clock**: The `turtlesim_node` publishes the turtle's pose at a high, steady frequency (approx. 60Hz).
- **Instant Reaction**: Every time a new message arrives on `/turtle1/pose`, the `rclpy.spin()` executor "wakes up" your `pose_callback`.
- **Synchronized Logic**: Your control logic runs at the exact same speed as the incoming sensor data.

Trigger Comparison

Feature	Timer-Based (Periodic)	Subscriber-Based (Event-Driven)
Trigger	CPU Internal Clock	External Sensor Message
Strategy	"Every 0.1s, move forward"	"When position changes, calculate move"
Best For	Heartbeats, Open-loop tasks	Reactive, Closed-loop control

Design Tip

By putting the `publish()` command inside the `pose_callback`, you ensure the robot never acts on "stale" (old) data. If the sensor stops, the controller stops.

This node acts as a "Brain" by combining both communication roles in one class.

- **Initialization**: We create both a subscription to listen to the state (`Pose`) and a publisher to send commands (`Twist`).

```
# Inside __init__
self.pose_subscriber_ = self.create_subscription(Pose, "/turtle1/pose",
                        self.pose_callback, 10)
self.cmd_vel_publisher_ = self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
```

- **Logic Loop**: The controller logic is triggered every time a new Pose message arrives.
- **Decision Making**: We use a simple `if-else` block to check if the turtle is near the "walls" (boundary coordinates).

Closed Loop Control

This is a reactive system: **Sense** (Subscriber) → **Think** (if-else logic) → **Act** (Publisher).

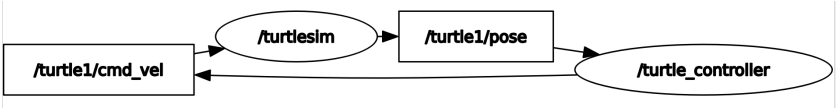


Figure: The Computational Graph for the closed-loop controller. Note how the node consumes one topic and produces another to affect the environment.

- **Dependency Management**: Since this node uses both `geometry_msgs` and `turtlesim` messages, both must be listed in your `package.xml`.
- **Execution**: Running `ros2 run pkg_name turtle_controller` will now automate the turtle's movement without manual teleop.

Verification and Debugging

Once your hybrid node is running, you can use `rqt_graph` to verify that the "circle" of communication is closed.

- **The Loop:** `/turtlesim` → `/turtle1/pose` → `/turtle_controller` → `/turtle1/cmd_vel` → `/turtlesim`.
- **Behavior:** The turtle should now change its behavior (e.g., turning) automatically when it approaches the walls.

Terminal Logic Check

```
# Monitor both topics at once in different terminals:
ros2 topic echo /turtle1/pose
ros2 topic echo /turtle1/cmd_vel
```

Control Frequency

In this example, the control rate is determined by the **incoming data** (the subscriber callback). If the Pose is published at 60Hz, your control loop also runs at 60Hz.

ROS 2 Services - Introduction

While Topics are for continuous streams, **Services** follow a **Request/Response** pattern. It is a synchronous "handshake" between a Client and a Server.

- **The Client**: Sends a request (e.g., "Spawn a new turtle at x=2, y=2").
- **The Server**: Processes the request and sends a response (e.g., "Turtle spawned successfully").

Service CLI Commands

Command	Result
`ros2 service list`	Lists all active services.
`ros2 service type /spawn`	Shows the interface used by the service.
`ros2 service call /reset std_srvs/srv/Empty`	Calls a service manually.

Blocking vs Non-blocking

Unlike Topics, a Service Client will often wait (block) until the result is returned or a timeout occurs.

Let's manually call a service provided by `turtlesim` to spawn a new turtle.

Spawn a new turtle

```
# Syntax: ros2 service call <service_name> <service_type> <arguments>
ros2 service call /spawn turtlesim/srv/Spawn "{x: 5.0,
y: 5.0, theta: 0.0, name: 'turtle2'}"
```

What happened?

1. You sent the (x, y, θ) coordinates to the `turtlesim` node.
2. The node created a second turtle.
3. It replied with the name of the new turtle.

Exercise

Try to use `ros2 service call /clear std_srvs/srv/Empty` to clean the drawing lines from the background!

When you spawn a second turtle, `turtlesim` doesn't mix the data. It creates a new **Namespace**.

- **Isolation:** `turtle1` and `turtle2` have their own separate "radio channels."
- **Topic List:** Run `ros2 topic list` after spawning. You will see:
  - `/turtle1/cmd_vel` and `/turtle2/cmd_vel`
  - `/turtle1/pose` and `/turtle2/pose`

Controlling the New Turtle

```
# This will NOT move turtle2
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}}"

# This WILL move turtle2
ros2 topic pub /turtle2/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}}"
```

Key Takeaway

In ROS 2, hardcoding names like `"/turtle1/..."` in your Python code makes it less flexible.

ROS 2 Services

A **Service** is a synchronous communication method. Unlike Topics, where data is broadcast regardless of who is listening, a Service only executes when specifically called.

- **Service Server**: The "Provider." It waits for a request, performs a task, and sends back a result. (e.g., The `/spawn` service in Turtlesim).
- **Service Client**: The "User." It sends a request and **blocks** (waits) until the response comes back.

When to use Services?

Use services for discrete actions that have a clear beginning and end:

- Changing a robot's mode (Auto vs. Manual).
- Triggering a camera snapshot.
- Calculating a complex inverse kinematics solution.

Just as Topics use `.msg` files, Services use `.srv` files. The structure is unique because it contains two distinct parts separated by three dashes (`---`).

Example: `AddTwoInts.srv`

Service Structure

```
int64 a
int64 b
---
int64 sum
```

- **Top Part**: The **Request**. What the Client sends to the Server.
- **The Divider**: Three dashes `---` separate the Request from the Response.
- **Bottom Part**: The **Response**. What the Server sends back to the Client.

Before writing code, we must know how to inspect the services available in the ROS 2 Graph.

- **List**: `ros2 service list`
- **Find Type**: `ros2 service type /spawn`
- **Inspect Structure**: `ros2 interface show turtlesim/srv/Spawn`

The /spawn service structure

```
float32 x
float32 y
float32 theta
string name # Optional
---
string name # Returns the name of the new turtle
```

Wait for Service

In code, a Client must always check if a Server is actually alive before calling it. We call this "Waiting for the service."

Just like Topics, every Service has a strictly defined API. Because services are a "handshake," the documentation is split into two parts.

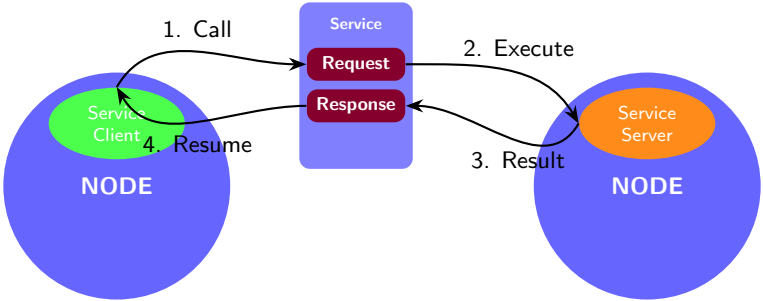
- **The Request**: Data sent from Client to Server.
- **The Response**: Data sent back from Server to Client.
- **Standard Library**: Most common robot services (e.g., `Empty`, `Trigger`, `SetBool`) are found in the `std_srvs` package.

Online API Sources

Package	Common Service	Documentation Link
`std_srvs`	`Empty`, `SetBool`	std_srvs API
`turtlesim`	`Spawn`, `Kill`, `SetPen`	turtlesim API

The "Three Dashes" Rule

When looking at the online API, look for the `---` separator.
Above the lines = What you must provide in your Python code.
Below the lines = What you will receive in your `future.result()`.



Synchronous Behavior

Note that the Client is "blocked" (waiting) between step 1 and step 4. It cannot perform other tasks until the Response is received.

Writing a Service Client follows a strict 4-step sequence to ensure the robot doesn't crash if the server is missing.

1. **Creation**: Define the Client object and link it to a specific service name and type.
2. **Discovery**: **Wait for the Server.** If you call a service that isn't "up" yet, the node will throw an error.
3. **Request Preparation**: Create a `Request` object and fill its fields (e.g., x, y coordinates).
4. **Asynchronous Call**: Send the request and wait for the **Future** (the placeholder for the upcoming response).

Why "Future"?

In Python, we call the service *asynchronously*. This means the node sends the message and continues to "spin" so it can receive the answer later without freezing the entire ROS 2 system.

Before coding, use the terminal to "interrogate" the service.

- **Find available services**:

```
ros2 service list
```

- **Check the Type**:

```
ros2 service type /spawn # Result: turtlesim/srv/Spawn
```

- **See the .srv Structure**:

```
ros2 interface show turtlesim/srv/Spawn
```

Reading the Interface

Everything **above** the --- is what your Client sends.
Everything **below** the --- is what your Client receives.

You will use these standard services during the laboratory exercises:

Turtlesim Services

Service	Type	Action
`/reset`	`std_srvs/srv/Empty`	Resets turtle to center.
`/clear`	`std_srvs/srv/Empty`	Clears all drawing lines.
`/kill`	`turtlesim/srv/Kill`	Removes a turtle.
`/spawn`	`turtlesim/srv/Spawn`	Creates a new turtle.
`/set_pen`	`turtlesim/srv/SetPen`	Changes line color/width.

The "Empty" Type

Some services (like `/reset`) don't need input and don't give output. They use the `Empty` type, which has nothing above or below the `---` lines.

In ROS 2, every process is a **Node**, but a Node can play different roles depending on the communication pattern.

- **The Node**: This is the "Container." It is the actual Python script running on your CPU.
- **The Server**: This is a **Functionality** hosted inside a Node.
- **One Node, Many Services**: A single node can host multiple servers (e.g., a "Robot" node might host a `/reboot` service and a `/get_status` service).

Analogy

Think of the **Node** as a Building and the **Server** as a Desk inside. One building can have a "Reception Desk," a "Help Desk," and a "Security Desk." You visit a specific desk to get a specific response.

Why can we call a service from the terminal? When you type a command like `ros2 service call`, ROS 2 isn't just sending a string; it is performing a complex operation:

1. **Hidden Node**: The command-line tool creates a **temporary, hidden node** on the fly.
2. **Discovery**: This hidden node looks for the Server in the Graph.
3. **The Handshake**: It sends the Request, waits for the Response, prints it to your screen, and then **dies** (the node is destroyed).

Debugging Power

This is why the CLI is so powerful: it allows you to simulate a "Client" node without writing a single line of Python code.

Unlike Topics (which are Many-to-Many), Services have specific rules about how they connect.

- **Many Clients → One Server**: This is the standard. Multiple nodes can all ask the `/turtlesim` node to `/reset` at the same time. The server processes them in order (queued).
- **One Client → Multiple Servers?**: **NO.** A service name must be unique. You cannot have two different nodes both hosting `/reset`. If you do, the Client won't know which one will answer (Non-deterministic).

Communication Comparison

Pattern	Topics	Services
Quantity	Many-to-Many	Many-to-One
Selection	Broadcast to all	Specific Request to one
Conflict	Allowed (but messy)	Not allowed (Service Name Collision)

Imagine a mobile robot with:

1. A **Safety Node** (Client)
2. A **User Interface** (Client)
3. A **Navigation Node** (Client)

All three can call the **Motor Controller** (Server) via a `/set_power_mode` service.

The Queue

If the Safety Node and the User call the service at the exact same millisecond, the Server's middleware handles the queuing. The Server processes one, sends the response, then starts the next.

A Minimal Client Node Template

```python
. . .
from turtlesim.srv import SetPen
from functools import partial

class TurtleControlNode(Node):
    . . .
    self.client = self.create_client(SetPen, "/turtle1/set_pen") #in init func.
    def pose_callback(self, pose: Pose):
        . . .
        if pose.x > 5.5 and not self.on_right_side:
            self.get_logger().info("Set color to red!")
            self.call_set_pen_service(255, 0, 0, 3, 0)
            self.on_right_side = True

        elif pose.x < 5.5 and self.on_right_side:
            self.get_logger().info("Set color to green!")
            self.call_set_pen_service(0, 255, 0, 3, 0)
            self.on_right_side = False
```

A Minimal Client Node Template

```
    . . .
    def call_set_pen_service(self, r, g, b, width, off):
        while not self.client.wait_for_service(1.0):
            self.get_logger().warn("Waiting for services...")

        request = SetPen.Request()
        request.r = r
        request.g = g
        request.b = b
        request.width = width
        request.off = off
        future = self.client.call_async(request)
        future.add_done_callback(partial(self.callback_set_pen))

    def callback_set_pen(self, future):
        try:
            response = future.result()
        except Exception as e:
            self.get_logger().error("Services call failed: %r" % (e,))
    . . .
```

When the Service response returns, ROS 2 calls your callback with exactly **one** argument: the `future`.

- **The Problem**: Sometimes you want your callback to know *which* turtle was changed or what the original request was.
- **The Solution**: `partial` allows you to "pre-fill" function arguments.

Using partial

```python
from functools import partial

# We 'wrap' the callback to include extra data
future.add_done_callback(partial(self.callback_func, extra_arg="red"))

def callback_func(self, future, extra_arg):
    # Now extra_arg is available here!
    self.get_logger().info(f"Finished setting color to {extra_arg}")
```

The Service Import

```python
from turtlesim.srv import SetPen
from functools import partial
```

- We import the **SetPen** service definition.
- `partial` is used to pass extra arguments to callbacks (very common in Python ROS 2).

Defining the Client

```python
client = self.create_client(SetPen, "/turtle1/set_pen")
while not self.client.wait_for_service(1.0):
    self.get_logger().warn("Waiting for services...")
```

- **Type**: It must match the Server (`SetPen`).
- **Waiting for Server**: The `while not client.wait_for_service(1.0)` loop is crucial. It pauses the node logic until the `turtlesim` node is actually ready to receive requests.

Why Wait?

If you call a service before the server is up, the request will disappear into the void. Always ensure the "Handshake" is possible before sending data.

Request and Async Call

```
request = SetPen.Request() # Initialize the Request object
request.r = r # Fill fields (RGB, Width)
...
future = self.client.call_async(request)
```

- **Async Call**: This sends the request and immediately moves to the next line of code. It does **not** wait for the turtle to finish changing color.
- **The Future**: This object is a "promise." It represents a result that hasn't arrived yet.

The Done Callback

```
future.add_done_callback(partial(self.callback_set_pen))
```

- This tells ROS 2: *"When the response finally arrives, run* `callback_set_pen` *automatically."*

Because ROS 2 is **Asynchronous**, we cannot wait for a response in the middle of our code. We use a `Future`.

- **Asynchronous Workflow**: `call_async()` returns immediately so your `pose_callback` can finish. This ensures the robot's movement logic is never interrupted.
- **Status Checking**: The Future knows if the server has replied yet.
- **Linking**: We use `add_done_callback` to tell ROS 2 which function to run once the "buzzer vibrates."

The "Future" Concept

A **Future** is a promise that a result will exist eventually. It allows the node to stay alive and responsive to sensor data while waiting for a slow service response.

In ROS 2, we use `call_async()` for services. This is because we cannot afford to stop the robot's brain while waiting for a response.

- **The Request**: Sent to the Server. The code immediately moves to the next line without waiting for an answer.
- **The Future**: A "placeholder" object returned by the call. It represents a result that hasn't arrived yet.
- **add_done_callback**: You "register" a function to be called later. You are telling ROS 2: *"I'm busy moving the robot; let me know when the response is ready!"*

Standard vs. Async

Type	Behavior	Effect on Robot
Synchronous	Wait for answer	Robot "freezes" (no new velocity commands).
Asynchronous	Use a "Future"	Robot continues processing Subscriber data.

The "Buzzer" Analogy

Calling a service asynchronously is like ordering food at a restaurant and receiving a **vibrating buzzer**. You don't stand at the counter staring at the chef; you go back to your table (the Subscriber callback) and continue your conversation. When the buzzer vibrates (the Future is done), you go get your result.

The Result Callback

```
def callback_set_pen(self, future):
    try:
        response = future.result()
    except Exception as e:
        self.get_logger().error("Service call failed: %r" % (e,))
```

- `future.result()`: This extracts the **Response** part of the `.srv` file.
- **Error Handling**: Service calls can fail (e.g., network timeout). Using a `try-except` block prevents your entire controller node from crashing if a service call goes wrong.

The Big Picture

In this node, your **Subscriber** triggers the **Service Client**. This is a classic "Event-Response" pattern: *"When I cross the center of the screen (Event), change my pen color (Response)."*

A **Synchronous Call** is the simplest way to interact with a service, but it comes with a high price: the node's "brain" stops until the answer arrives.

- **Mechanism**: When you use `client.call(request)`, the current thread is "suspended."
- **Blocking**: No other callbacks (Subscribers, Timers) can run while the thread is waiting.
- **The Deadlock Risk**: If the response arrives but the thread is still "waiting," the node cannot process the response. **The node freezes forever.**

Synchronous Call Example (Use with caution!)

```
#This logic is only safe in simple scripts without other callbacks

request = SetPen.Request()
request.r = 255
... fill other fields ...

#THE BLOCKING LINE: Node stops here
response = self.client.call(request)

self.get_logger().info("Service finished successfully!")
```

When to use it?

Synchronous calls are acceptable **only** in startup scripts or one-off "Task" scripts where the node is not handling sensor data or multiple topics simultaneously.

In the lab, you must decide: do I want my robot to "freeze" or "keep thinking"?

Call Comparison

Feature	Synchronous (`.call`)	Asynchronous (`.call_async`)
Logic Flow	Linear (Step-by-step)	Event-Driven (Parallel)
Complexity	Low (Easy to read)	High (Requires "Future" + Callback)
Safety	**Low**: Can cause deadlocks	**High**: Node stays responsive
Usage	Simple Setup Scripts	Real-time Robot Controllers

- **Solution**: Always use the **Future** object and `call_async` for interactive robotics.

Hands-on

Goal: Create a node that makes the turtle draw a continuous circle.

1. Create a new file `draw_circle.py` in your package.
2. Use a **Publisher** on the topic `/turtle1/cmd_vel`.
3. **Task:** Set a constant `linear.x` of 2.0 and an `angular.z` of 1.0.
4. Build, source, and run.

Challenge

Modify the code so the turtle stops after 5 seconds or draw different shapes.

Goal: Combine a Subscriber and a Service Client.

1. Create `color_controller.py`.
2. **Subscribe** to `/turtle1/pose`.
3. **Logic:**
   - If the turtle is on the **left** half of the screen ($x < 5.5$), call the `/set_pen` service to make the pen **Green**.
   - If it is on the **right** half ($x > 5.5$), make it **Red**.
4. **Test:** Run your `draw_circle` node from Exercise 1 and this node simultaneously.

Checklist

Did you add `turtlesim` and `geometry_msgs` to your `package.xml`?
Did you add the entry point to `setup.py`?

Goal: Use the CLI to interact with a running system.

1. While your turtle is drawing, open a new terminal.
2. Use the `teleport_absolute` service to jump the turtle to the center:

```
ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 5.5, y: 5.5, theta: 0.0}"
```

3. Observe the "Teleop" behavior: Does the turtle stop drawing?